

JSSDL 项目结构说明

基于联合与特定共享字典学习的工业过程建模与监测

项目目录树

jssdl/	
— README.md	# 项目说明与快速上手指南
— requirements.txt	# 依赖包列表
— config.yaml	# 全局超参数配置文件
— data/	# 数据目录（不纳入版本控制）
— raw/	# 原始数据
— numerical/	# 数值仿真数据（由脚本生成）
— TEP/	# Tennessee Eastman 过程数据
— d00.dat	# 正常训练集
— d00_te.dat	# 正常测试集
— d01.dat ~ d21.dat	# 21种故障测试集
— roasting/	# 焙烧过程真实工业数据
— train_normal.csv	# 正常训练样本（1000条）
— test.csv	# 测试样本（500正常+500故障）
— processed/	# 预处理后数据（归一化、分割等）
— numerical/	
— TEP/	
— roasting/	
— jssdl/	# 核心算法包
— __init__.py	
— model/	# 模型核心模块
— __init__.py	
— jssdl.py	# ★ JSSDL 主模型类 (Algorithm 1)
— dictionary_update.py	# 字典更新子函数
— # 包含:	
— # update_D1()	— 特定字典更新 (Eq.16, 几何投影法)
— # update_P()	— 辅助矩阵更新 (Eq.19, SVD软阈值)
— # update_D2()	— 共享字典更新 (Eq.21, 闭式解)
— sparse_coding.py	# 稀疏编码模块
— # 包含:	
— # omp_encode()	— OMP算法封装
— # update_X1_X2()	— 三步更新法 (Eq.23-25)
— soft_threshold.py	# 软阈值算子 $S_{\tau}(\sigma)$ (Eq.18)

```

|   └─ monitoring/                                # 过程监测模块
|   |   └─ __init__.py
|   |   └─ offline.py                            # 离线建模阶段
|   |   |   # 包含:
|   |   |   #   compute_train_errors() - 计算训练重构误差  $R_j$  (Eq.26)
|   |   |   #   kde_threshold()       - KDE估计控制限  $R_{tr}$  (Eq.27)
|   |   └─ online.py                             # 在线监测阶段
|   |   |   # 包含:
|   |   |   #   encode_new_sample()    - 新样本稀疏编码 (Eq.28)
|   |   |   #   compute_jre()          - 联合重构误差 JRE (Eq.29)
|   |   |   #   detect_fault()         - 故障判决 (JRE vs  $R_{tr}$ )
|   |   └─ metrics.py                          # 评估指标
|   |   |   # 包含:
|   |   |   #   compute_fdr()          - 故障检测率 (Eq.30)
|   |   |   #   compute_far()         - 误报率 (Eq.31)
|   |
|   └─ utils/                                    # 工具函数
|       └─ __init__.py
|       └─ data_loader.py                       # 数据加载与预处理 (归一化等)
|       └─ initializer.py                      # 字典随机初始化工具
|       └─ visualizer.py                       # 结果可视化 (监测图、热力图、收敛曲线)
|
|─ experiments/                                # 实验脚本 (对应论文 Section III)
|   └─ exp1_numerical.py                       # 实验1: 数值仿真 (Section III-A)
|   |   # 生成合成数据 → 训练JSSDL → 与PCA/RobustPCA/DL对比 → 输出Fig.6
|   └─ exp2_TEP.py                            # 实验2: TEP过程 (Section III-B)
|   |   # 21种故障逐一测试 → 生成Table I FDR对比表
|   └─ exp3_roasting.py                       # 实验3: 焙烧过程 (Section III-C)
|   |   # 工业真实数据验证 → 输出Fig.10监测图
|   └─ sensitivity_analysis.py                 # 参数敏感性分析 ( $\tau$  与 D1稀疏度, 对应Fig.5)
|
|─ baselines/                                  # 对比基线方法
|   └─ __init__.py
|   └─ pca_monitor.py                          # PCA ( $T^2$  + SPE)
|   └─ robust_pca_monitor.py                   # Robust PCA
|   └─ dl_monitor.py                           # 传统单字典学习 (DRE)
|
|─ tests/                                       # 单元测试
|   └─ test_soft_threshold.py                  # 测试软阈值算子正确性
|   └─ test_dictionary_update.py               # 测试各字典更新步骤
|   └─ test_omp.py                             # 测试OMP稀疏编码
|   └─ test_monitoring_pipeline.py             # 测试端到端监测流程
|
|─ notebooks/                                  # Jupyter可视化分析笔记本
|   └─ 01_data_exploration.ipynb              # 数据特性分析 (复现Fig.1、Fig.7)
|   └─ 02_algorithm_walkthrough.ipynb        # 算法逐步演示与中间量可视化
|   └─ 03_results_visualization.ipynb        # 实验结果图表复现

```

```
|
└─ outputs/                                # 输出目录（自动生成，不纳入版本控制）
    └─ figures/                            # 监测图、热力图、收敛曲线
    └─ tables/                             # FDR/FAR 对比表（CSV格式）
    └─ checkpoints/                       # 训练好的字典（D1, D2）保存
```

📄 各模块详细说明

1. `jssdl/model/jssdl.py` — 主模型类 ★

这是整个项目的核心，实现论文算法1。

类 JSSDL:

属性:

- `n_atoms_D1, n_atoms_D2` 字典原子数
- `sparsity_D1` D1 的原子稀疏度参数 a (Eq.16)
- `sparsity_X1, sparsity_X2` 编码稀疏度 (OMP的 T_0)
- `lambda_1~5` 目标函数 5 个正则化权重 (Eq.10)
- `tau` SVD 软阈值参数 τ (Eq.18-19), 控制 D2 的秩
- `max_iter` 最大迭代次数

方法:

- `fit(Y)` 离线训练 (执行 Algorithm 1 主循环)
- `_update_D1()` Step 1: 更新特定字典 (Eq.16)
- `_update_P()` Step 2: 更新辅助矩阵 (Eq.19, SVD+软阈值)
- `_update_D2()` Step 3: 更新共享字典 (Eq.21, 闭式解)
- `_update_X1_X2()` Step 4: 三步OMP更新编码 (Eq.23-25)
- `_compute_objective()` 计算目标函数值 (用于收敛判断)
- `set_threshold(Y, alpha)` 用KDE在训练数据上确定控制限 R_{tr} (Eq.27)
- `predict(y_new)` 在线检测: 返回 JRE 值 (Eq.28-29)
- `is_fault(y_new)` 在线检测: 返回布尔故障判决

2. `jssdl/model/dictionary_update.py` — 字典更新子函数

对应Section II-B中三个字典/矩阵的更新步骤:

函数	回复公式	说明
<code>update_D1(Y1, X1, lambda2, a)</code>	公式16	逐列更新, 基于几何投影的稀疏闭式解
<code>update_P(D2, tau)</code>	公式19	SVD分解后对奇异值做软阈值收缩
<code>update_D2(Y, D1, X1, X2, P, lambda5)</code>	公式21	对D2求偏导令其稳态得闭式解

重点: `update_D1`中的`Sort`排序逻辑——对 $\|e_{ji}\|_1$ 从大到小排序, 只保留排名 $\leq a$ 的元素对应的`d_ji`非零。

3.——`jssdl/model/sparse_coding.py`稀疏编码 (OMP)

实现Eq.23-25的三步解耦更新:

```
Step 1: X2 ← OMP(Y, D2, T2)           # 先用 D2 重构全局共性
Step 2: X1 ← OMP(Y - D2·X2, D1, T1)    # 再用 D1 重构残差 (特性)
Step 3: X2 ← OMP(Y - D1·X1, D2, T2)    # 重新用 D2 重构剩余共性
```

OMP可调用`sklearn.linear_model.orthogonal_mp`或手动实现。

4.`jssdl/monitoring/`——过程监测模块

离线阶段(`offline.py`):

- 完成训练后, 对所有训练样本计算重构 R_j (Eq.26)
- 用Gaussian KDE 移植分配, 按置信水平 α (如0.99) 确定控制限 R_{tr} (Eq.27)
- 可行的`scipy.stats.gaussian_kde`实现

在线阶段(`online.py`):

- 新样本到达→图纸Eq.28 (固定 D_1 、 D_2 , 对 x_1 、 x_2 做带稀疏约束的最优化)
 - 计算JRE (Eq.29) →与 R_{tr} 比较→输出正常/故障
-

5.`config.yaml`——超参数配置

yaml

```
numerical_simulation:
    n_atoms_D1: 100
    n_atoms_D2: 100
    sparsity_D1: 2          # 论文最优参数
    sparsity_X1: 2
    sparsity_X2: 3
    tau: 0.18              # 软阈值，控制 D2 的秩
    kde_confidence: 0.92
    max_iter: 100

TEP:
    n_atoms_D1: 100
    n_atoms_D2: 100
    sparsity_D1: 5
    sparsity_X1: 5
    sparsity_X2: 5
    tau: 0.08
    kde_confidence: 0.99
    max_iter: 100

roasting:
    n_atoms_D1: 100
    n_atoms_D2: 100
    sparsity_D1: 2
    sparsity_X1: 4
    sparsity_X2: 2
    tau: 0.01
    kde_confidence: 0.99
    max_iter: 100
```

6. experiments/ — 三个实验脚本

exp1_numerical.py (数值仿真，对应图6)

1. 按Eq.32-33生成数据 $y = k_1 \cdot y_1 + k_2 \cdot y_2 + e$ 训练：，1000条 + 1000条测试
2. 故障：在 y_7 维度总部+10的偏差故障（后500条）
3. 训练JSSDL及长江基线，计算FDR/FAR，缺量监测图

exp2_TEP.py (TE过程，对应表一)

1. 加载标准TEP数据集（33个变量：22连续+11个操作）
2. 对21种故障逐一测试，故障在第160个样本后导入
3. 输出FDR对比表（重点关注IDV 5, 16, 19, 20）

`exp3_roasting.py` (烘焙过程, 对应图10)

- 1. 加载 25 维工业数据 (见表二)
- 2. 训练集: 1000条正常样本; 测试集: 500条正常+500条欠氧化故障
- 3. 对比各方法FDR和FAR, 重点验证JSSDL的低误报率

`sensitivity_analysis.py` (参数结构, 对应Fig.5)

- 遍历 $\tau \in [0.10, 0.19]$ 和 D1稀疏度 $\varepsilon \in [2, 20]$
- 记录D2的Rank、FAR和FDR, 上下文三维曲面图

7. `baselines/` — 对比基线

文件	方法	统计量
<code>pca_monitor.py</code>	主成分分析	T^2 和 SPE
<code>robust_pca_monitor.py</code>	稳健主成分分析 (RPCA)	rT^2 和 $rSPE$
<code>dl_monitor.py</code>	传统字典学习	DRE (字典重构)

🔑 关键实际注意事项

- 1. `update_D1` 的排序逻辑: $\|e_{ji}\|_1$ 是对差矩阵 E_j 中第 i 行的 $L1$ 范数, 非残排序后只让排名靠前 (变化最大) 的一个元素对应的 d_{ji} 取零值。
- 2. τ 的调节: τ 直接控制 D2 的排序。 τ 过大 $\rightarrow$ D2 排序过低 $\rightarrow$ 共性变得过度粗糙 $\rightarrow$ JRE 持续偏高; τ 过小 $\rightarrow$ D2 排序过高 $\rightarrow$ 共性空间突变 $\rightarrow$ FAR 上升。
- 3. 三步编码解耦合: Eq.23-25 的顺序不可颠倒, 先提共性、再提特性、再修正共性, 是保证两个字典语义分离的关键。
- 4. 初始化: 论文策略建议多次随机初始化取平均FDR/FAR, 因此实验脚本中应设置 `n_runs` 参数。
- 5. D2 更新的矩阵求逆: Eq.21 中 $(X_2 \cdot X_2^T + \lambda \cdot 5 \cdot E)$ 需要加正则项 $\lambda \cdot 5 \cdot I$ 阻止奇异数, E 即单位矩阵。

📦 依赖包 (`requirements.txt`)

numpy>=1.24

```
scipy>=1.10  
scikit-learn>=1.3  
matplotlib>=3.7  
pandas>=2.0  
pyyaml>=6.0  
jupyter>=1.0  
tqdm>=4.65
```

快速运行示例

狂欢

1. 安装依赖

```
pip install -r requirements.txt
```

2. 运行数值仿真实验

```
python experiments/exp1_numerical.py
```

3. 运行 TE 过程实验（需先下载 TEP 数据集）

```
python experiments/exp2_TEP.py
```

4. 参数敏感性分析

```
python experiments/sensitivity_analysis.py
```

5. 查看交互式演示

```
jupyter notebook notebooks/02_algorithm_walkthrough.ipynb
```